

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import minimize
from numpy.linalg import lstsq, cond, matrix_rank

import sympy as sp

%matplotlib inline
```

```
In [2]: #Question 1A

np.random.seed(43201)

x = np.random.normal(0,1, size = (100,5))

X = np.column_stack((np.ones(100), x))

B = np.array([3,1.4,-0.9,0.7,0,1.1]).T

e = np.random.normal(0,0.7, size=100)

y = X@B + e

shape_x = X.shape

print(X.shape)

rank_x = np.linalg.matrix_rank(X)

b_hat,residuals,rank,singular_values = np.linalg.lstsq(X,y)

print(rank_x)

print(b_hat)

print(B)

#when we compare the estimated coeffcients to the true values used
#generate y are close but not identical and that is known from the
#error values as part of the estimatec oeffcients and so overall it is good
#estimate

(100, 6)
6
[ 2.87103129e+00  1.37224778e+00 -9.81646932e-01  5.24678012e-01
  1.06857574e-03  1.06353502e+00]
[ 3.    1.4 -0.9  0.7  0.    1.1]
```

```
In [3]: #Question 1B

y_hat = X@b_hat
```

```

r = y - X@b_hat

RMSE = (np.mean(r**2))**1/2

print(y_hat)
print(r)
print(RMSE)

```

```

[ 3.392203    2.46218    3.30497587  4.34635647 -0.34718828  1.41657933
 2.30782041  1.88977854  4.11114724  3.39044322  4.37300767  3.31911023
 2.75546007  3.28917084  2.17474573  1.86892479  0.1927826   3.15376378
 1.95111536  3.17835581  1.84502547  4.91200163  3.15645897  1.51280461
 0.36076316  0.41659101  0.15744281 -0.15269019  3.26074704  2.1189702
 2.89147976  2.03301315  4.06078123  4.60379634  6.61947156  1.64561435
 4.330285    6.86618134  2.76529099  0.74339789  5.30613911  3.01570362
 1.90435579  3.36587444  0.92434114  4.33645693  3.23270541  4.83800773
 6.49029923  3.03247135  5.66022528  0.61655645  4.58374811  4.57005351
 0.98823172  2.24076533  1.07028905  1.95539075  1.00388826  3.65785089
-0.01907116  3.18766599 -0.27927448  1.5333979   5.39436498 -0.20800494
 0.60581394 -1.87827103  4.18503164  3.86522595  3.63208559  1.8586632
 0.96398888  2.51902357  4.08849557 -1.07359162 -0.7167925   5.29388428
 2.95015606 -0.22878742  4.94778697  6.15112062  5.11758997  2.00641354
 1.29985991  0.26525514  3.43611215  4.45591994  2.90642039  1.60933756
 1.04465759  5.19559258  3.33166841  1.87138067 -0.19202254  3.28157396
 5.62699872  3.15496489  1.22617987  2.85569265]
[-0.5006352   0.39160595 -0.99157035  0.73955053  0.79705567  0.5988258
-0.89671062  0.51030061 -0.13355809  0.08308044 -1.08375142  0.35578555
-0.55253911 -0.43802292 -0.32621352 -0.82076553 -0.7071883   0.14116524
 0.36689903 -0.98538528 -1.16426551 -0.1709138   0.40481734  0.75887864
 0.13997317  0.76412227  0.66463911 -1.53523777 -0.50757231  0.06666716
 0.52899465 -0.1198948  -1.18711244 -1.08296876 -0.11618355 -0.16907987
-0.36445694  0.06922536  0.31877937 -0.31991746  0.47780961  0.29892383
 0.15949493  0.81737054 -0.06333193  0.57331851 -0.52944507 -0.16948051
 1.21502533  1.62622788  0.28608445 -0.11623295  0.70649337  0.37483785
 0.80863219  1.70539178  0.23204066  1.41167165 -1.84025819  0.54341872
-0.50586722 -0.3812736   0.47783215  0.3944708   -0.84107538  1.51783582
 0.18224497  1.08146498  2.50986165  0.6083527   -1.94138734 -0.37881968
-0.41103317 -0.21171096 -0.07439164 -0.45783299  0.91884028 -0.29005078
 0.87371945  0.00303846  1.21671178  0.13396445 -1.14592009  0.45570051
-0.3620631  -0.77574928 -0.32949525 -1.11814652 -1.1691207  -0.59447205
-0.22831356  1.31294412  0.63888144 -0.1111773  -1.23600037 -1.359101
-0.06293391 -1.05418135  0.0636956   0.60614312]
0.3233027623350673

```

In [4]: *#Question 1c*

```

matrix = X.T @ r

inf_norm = np.linalg.norm(matrix , ord = np.inf)

print(inf_norm)

```

```
#the norm is close to  
#zero because this test square residuals are orthogonal to every column of X
```

1.545430450278218e-13

In [5]: *#Question 2a*

```
# the reason maximizing the log likelyhood along B is equal residual sum of  
#squares because the second two terms of log likelyhood depend on B and those  
#form the residual sum of squares formula but the negative version so its the  
#so if I want to minimize the RSS we need to make sure we maximize the log  
likelihood  
  
#we can B is the same B_hat because both minimize the RSS
```

In [6]: *#Question 2b*

```
n_sym, k_sym = sp.symbols('n k')  
sigma2 = sp.symbols('sigma2')  
  
y_sym = sp.MatrixSymbol('y', n_sym, 1)  
X_sym = sp.MatrixSymbol('X', n_sym, k_sym)  
B_sym = sp.MatrixSymbol('B', k_sym, 1)  
  
residual = y_sym - X_sym * B_sym  
  
ell = -(n_sym / 2) * sp.log(2 * sp.pi * sigma2) - ((residual.T * residual) / (2 * sigma2))  
  
d_ell = sp.diff(ell, sigma2)  
sigma2_mle = sp.solve(sp.Eq(d_ell, 0), sigma2)[0]  
  
print("d ell / d sigma^2 =", sp.simplify(d_ell))  
print("sigma^2 MLE (symbolic) =", sigma2_mle)  
  
n = len(y)  
k = X.shape[1]  
rss = float(np.sum(r**2))  
  
sigma2_mle_num = rss / n  
sigma2_unbiased = rss / (n - k)  
  
print("sigma^2 MLE =", sigma2_mle_num)  
print("sigma^2 unbiased =", sigma2_unbiased)  
  
# the denominator is different because in unbiased version in both estimates  
#as num but unbiased version we used 6 estimators or 6 degree of freedom so  
#independent information so that's why they're different
```

```

d ell / d sigma^2 = (-n + Sum(y[_i_1, 0]**2 - y[_i_1, 0]*Sum(B[_i_2, 0]*X[_i_1, _i_2], (_i_2, 0, k - 1)) - y[_i_1, 0]*Sum(B[_i_3, 0]*X[_i_1, _i_3], (_i_3, 0, k - 1)) + Sum(B[_i_2, 0]*X[_i_1, _i_2], (_i_2, 0, k - 1))*Sum(B[_i_3, 0]*X[_i_1, _i_3], (_i_3, 0, k - 1)), (_i_1, 0, n - 1))/sigma2)/(2*sigma2)
sigma^2 MLE (symbolic) = Sum(y[_i_1, 0]**2 - y[_i_1, 0]*Sum(B[_i_2, 0]*X[_i_1, _i_2], (_i_2, 0, k - 1)) - y[_i_1, 0]*Sum(B[_i_3, 0]*X[_i_1, _i_3], (_i_3, 0, k - 1)) + Sum(B[_i_2, 0]*X[_i_1, _i_2], (_i_2, 0, k - 1))*Sum(B[_i_3, 0]*X[_i_1, _i_3], (_i_3, 0, k - 1)), (_i_1, 0, n - 1))/n
sigma^2 MLE = 0.6466055246701345
sigma^2 unbiased = 0.6878782177341858

```

In [7]: *#Question 2c*

```

e_x = np.array([0,0,1,0,0,0]).T

def log_likelihood(B, sigma_2):
    n = len(y)
    residual = y - X @ B
    rss = residual.T @ residual
    return -(n / 2) * np.log(2 * np.pi * sigma_2) - rss / (2 * sigma_2)

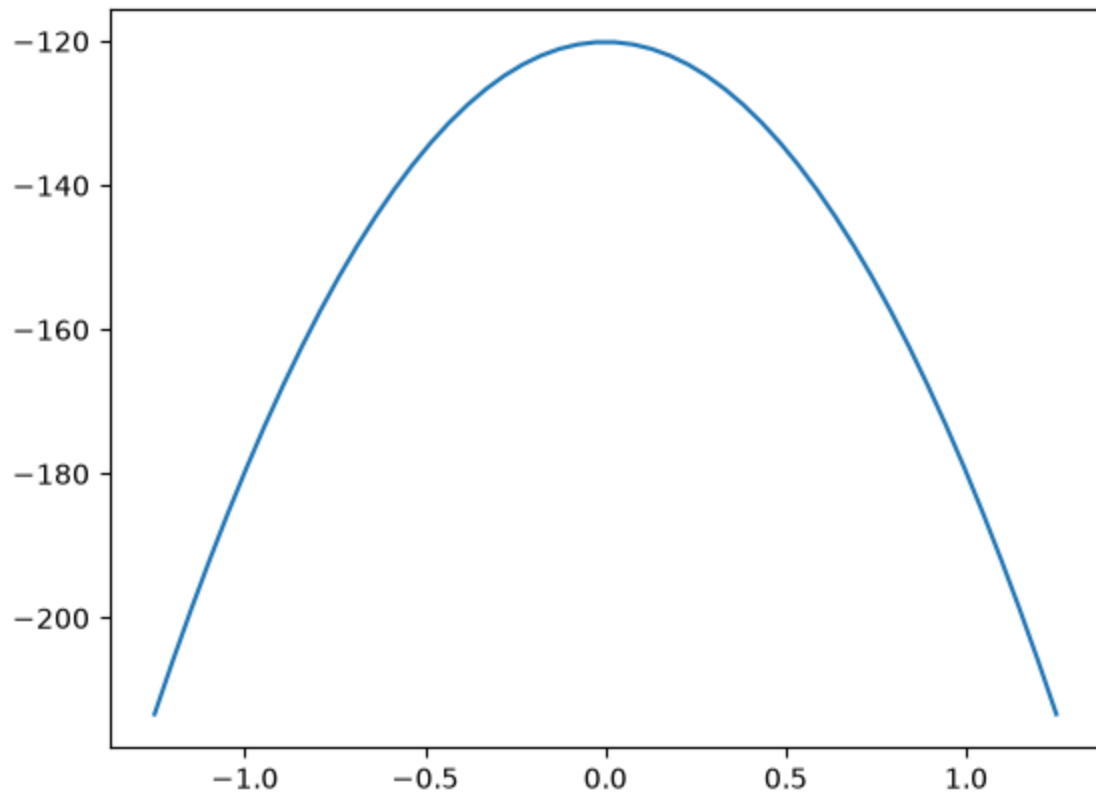
t_values = np.linspace(-1.25, 1.25, 50)

y_values = np.array([
    log_likelihood(b_hat + t * e_x, sigma2_mle_num)
    for t in t_values
])

plt.plot(t_values, y_values)

```

Out[7]: [*<matplotlib.lines.Line2D at 0x112fca710>*]

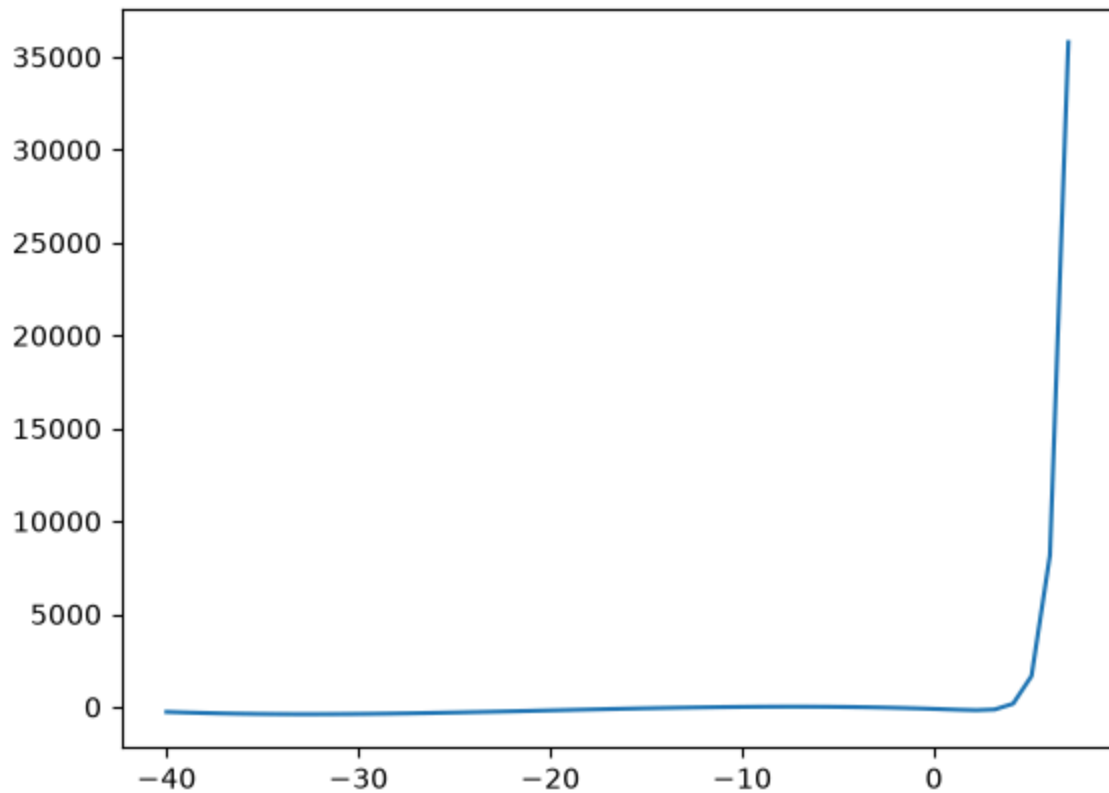


In [8]: *#Question 3a*

```
def f(x):
    return np.exp(1.5*x) - 3*(x+6)**2 - 0.05*x**3

x_values = np.linspace(-40, 7, 50)
y_values = f(x_values)
plt.plot(x_values, y_values)
plt.show()

#there are mutiple local min and maxmiums and most of curve is flat
#and it increases very suddenly and rapidly all of which could effect opotimi
```



In [9]: *#Question 3b*

```
def f_prime(x):
    return 1.5*np.exp(1.5*x) - 6*(x+6) - 0.15*x**2

value_start_at_negative_15 = minimize(f, x0 = [-15], jac = f_prime, method = "BFGS")

print(value_start_at_negative_15.x)
print(value_start_at_negative_15.fun)
print(abs(f_prime(value_start_at_negative_15.x[0])))
print(value_start_at_negative_15.success)

value_start_at_0 = minimize(f, x0 = [0], jac = f_prime, method = "BFGS")

print(value_start_at_0.x)
print(value_start_at_0.fun)
print(abs(f_prime(value_start_at_0.x[0])))
print(value_start_at_0.success)
```

```
[-32.64911064]
-390.38577025077666
2.842170943040401e-14
True
[2.34996709]
-175.86262017122644
8.126811246289556e-06
True
```

In [10]: #Question 3c

```
#the reason different starting values can converge to different answers bec  
#a each starting value can be either before minium which it it was decreasin  
#in different calculation or if after minmium could be increasing with diffe  
#the converge that started at 0 has object lower value and I need more evi  
#is not other minmium lower in that interval or try with other starting poin
```

In [11]: #Question 4a

```
np.random.seed(43202)  
  
u = np.random.normal(0, 1, size=n)  
  
x1 = X[:, 1]  
x6 = x1 + 1e-4 * u  
  
X_plus = np.column_stack((X, x6))  
  
y_star = y + 1e-3 * u  
  
print("Shape of X_plus:", X_plus.shape)  
print("Shape of y_star:", y_star.shape)  
  
y_x = X@B + e  
  
b_hat_plus, _, _, _ = lstsq(X_plus, y, rcond=None)  
  
r_x = y_x - X@b_hat  
  
r_plus = y_x - X_plus@b_hat_plus  
  
print(cond(X))  
  
print(cond(X_plus))  
  
RMSE_x = (np.mean(r_x**2))**(1/2)  
  
RMSE_x_plus = (np.mean(r_plus**2))**(1/2)  
  
print(RMSE_x)  
  
print(RMSE_x_plus)  
  
print(b_hat_plus[1])  
  
print(b_hat_plus[6])
```

```
Shape of X_plus: (100, 7)
Shape of y_star: (100,)
1.5162978823413717
23353.001703373957
0.8041178549628994
0.8009794381238184
-782.7377812425848
784.1078233396371
```

In [12]: *#Question 4B*

```
b_hat_plus_star, _, _, _ = lstsq(X_plus, y_star, rcond=None)

print(b_hat_plus_star[1] - b_hat_plus[1])

print(b_hat_plus_star[6] - b_hat_plus[6])

y_hat_plus = X_plus @ b_hat_plus

y_hat_plus_star = X_plus @ b_hat_plus_star

print(np.sqrt(np.mean((y_hat_plus - y_hat_plus_star)**2)))

print(np.max(np.abs(y_hat_plus - y_hat_plus_star)))
```

```
-9.999999999999886
9.999999999999886
0.0009351217032457578
0.002622779196128633
```

In [13]: *#Question 4c*

```
#the reason fitted values don't change a lot is because x1 and x6 when subtr
#equal close to 0 because of there colinearity which results regardless of w
#by it still results 0 or fitted values don't change but the coeffcients do
#so it hard to predict or see changes in coeffcients with a specfic change b
#the predictors don't change
```

In [14]: *#Question 5a*

```
df = pd.read_csv("data/data-manipulation.csv")

print(df.shape)

print(df.dtypes)

print(df["observation_id"].duplicated().sum())

print(df.isna().sum())

new_df = df.dropna(subset=["response"]).copy()

new_df["feature_sum"] = new_df["x1"] + new_df["x2"]
```



```
(24, 5)
observation_id      str
group               str
x1                  float64
x2                  float64
response            float64
dtype: object
0
observation_id      0
group               0
x1                  0
x2                  0
response            3
dtype: int64
```

In [15]: *#Question 5b*

```
total_sum = new_df.groupby("group").agg(
    observations=("response", "size"),
    mean_response=("response", "mean"),
    mean_feature_sum=("feature_sum", "mean"),
)

print(total_sum)
```

	observations	mean_response	mean_feature_sum
group			
A	7	5.600000	3.600000
B	7	7.602857	4.928571
C	7	7.997143	6.585714

In [16]: *#Question 5c*

```
sorted_group = new_df.sort_values("response", ascending=False)

three = sorted_group[["observation_id", "group", "response", "feature_sum"]]

print(three)

print(len(new_df) == df["response"].notna().sum())

print(new_df["response"].isna().sum() == 0)

print(new_df["observation_id"].is_unique)
```

	observation_id	group	response	feature_sum
21	obs-22	C	9.24	6.6
15	obs-16	B	9.00	5.2
13	obs-14	B	8.66	5.0

True
True
True

In [17]: *#Question 6a*

#the 39 million records are not independent because it may include records t

#same participants across multiple days and if all recorded data
#are averaged together, participants with more recorded days would have more weight
#or skew the average higher than a normal person and it will be representative
#of people who wore the watch or only a few recorded days

In [18]: #Question 6b

No it does not because the two groups were not formed by random assignment
#people in BYOD already had a fitbit so they are different from WEAR people
#like activity level, income, motivation, etc and these differences
#may result in step difference

In [19]: #Question 6c

#The dataset can still be biased because while it is large
#but the participants first of all weren't randomly selected and the recorded
#data were already pre-selected and people who joined this
#research study maybe different from the people who represent
#an average American and all since only certain days are used for the recording
#it does not showcase a walking style on every other day because some people may
#on some days